RAJALAKSHMI ENGINEERING COLLEGE

Yukesh R 251001316-2025-IT

Y2

Started on	Wednesday, 30 September 2026, 9:35 AM
State	Finished
Completed on	Wednesday, 30 September 2026, 9:36 AM
Time taken	57 secs
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 10.00 out of 10.00

Playing with Chessboard:

Ram is given with an n*n chessboard with each cell with a monetary value. Ram stands at the (0,0), that the position of the top left white rook. He is been given a task to reach the bottom right black rook position (n-1, n-1) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can only travel one step right or one step down the board. Help ram to achieve it by providing an efficient DP algorithm.

Example:

Input

3

1 2 4

2 3 4

8 7 1

Output:

19

Explanation:

Totally there will be 6 paths among that the optimal is

Optimal path value:1+2+8+7+1=19

Input Format

First Line contains the integer n

The next n lines contain the n*n chessboard values

Output Format

Print Maximum monetary value of the path

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

// Helper macro to find maximum of two values
#define MAX(a, b) ((a) > (b) ? (a) : (b))

int main() {
    int n;

    // Read the size of the chessboard
    if (scanf("%d", &n) != 1 || n <= 0) {
        return 0;
    }

    // Allocate memory for a 1D DP table representing a single row
    long long *dp = (long long *)calloc(n, sizeof(long long));
    if (dp == NULL) {
        return 1;
    }

    // Read the chessboard data and build the DP values row by row
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            long long val;
            scanf("%lld", &val);

            if (i == 0 && j == 0) {
                // Base Case: Starting Position
                dp[j] = val;
            } else if (i == 0) {
                // First Row: Can only come from the left (dp[j-1])
                dp[j] = val + dp[j - 1];
            } else if (j == 0) {
                // First Column: Can only come from above (dp[j] contains the above cell's total)
                dp[j] = val + dp[j];
            } else {
```

```
                // Inner Cells: Maximum of coming from above (dp[j]) or left (dp[j-1])
                dp[j] = val + MAX(dp[j], dp[j - 1]);
            }
        }
    }

    // The last element contains the maximum monetary value to reach (n-1, n-1)
    printf("%lld\n", dp[n - 1]);

    // Free allocated memory
    free(dp);

    return 0;
}
```

	Input	Expected	Got	
✔	3 1 2 4 2 3 4 8 7 1	19	19	✔
✔	3 1 3 1 1 5 1 4 2 1	12	12	✔
✔	4 1 1 3 4 1 5 7 8 2 3 4 6 1 6 9 0	28	28	✔

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

Back to Course